Notebook 48: Song-Level Genre Prediction with Confidence Scores

Purpose

This notebook tests the final audio genre prediction pipeline on one song or audio file.

It loads the audio, splits it into multiple 15-second windows, runs each window through the final hybrid model, and then averages the results to produce a song-level prediction.

What This Notebook Does

1. Loads an external audio file or FMA track.
2. Loops short audio if needed.
3. Splits the audio into multiple windows.
4. Predicts genres for each window.
5. Averages the hybrid confidence scores.
6. Selects the highest-scoring genre as the main predicted genre.
7. Shows other strong genres as secondary or related predictions.
8. Applies safer Stage 2 rare-tail fallback logic.
9. Saves the final prediction outputs.

How to Read the Output

The genre with the highest average hybrid confidence score is treated as the main predicted genre.

Other genres above the secondary threshold are shown as secondary or related genres.

The confidence score is a model prediction score, not accuracy. It shows how strongly the model associates the audio with that genre based on the FMA-trained pipeline.

Final Note

This notebook is for practical testing of the completed pipeline. Results are most reliable for audio similar to the FMA dataset, and external commercial or phone-recorded audio should be interpreted carefully.

In [1]:
```
# ============================================================
# 0. INSTALL / CHECK REQUIRED PACKAGES
```

```
# ============================================================

import sys
import importlib.util
import subprocess

required_packages = {
    "joblib": "joblib",
    "librosa": "librosa",
    "numpy": "numpy",
    "pandas": "pandas",
    "scipy": "scipy",
    "sklearn": "scikit-learn",
    "soundfile": "soundfile",
    "audioread": "audioread",
    "tensorflow": "tensorflow==2.20.0"
}

missing_packages = []

for import_name, pip_name in required_packages.items():
    if importlib.util.find_spec(import_name) is None:
        missing_packages.append(pip_name)

if missing_packages:
    print("Installing missing packages:", missing_packages)
    subprocess.check_call([
        sys.executable,
        "-m",
        "pip",
        "install",
        *missing_packages
    ])
else:
    print("All required packages are already installed.")

print("Python executable being used:")
print(sys.executable)
```

```
All required packages are already installed.
Python executable being used:
e:\SCHOOL\Masters\Capstone_FMA_Project\notebook\.venv\Scripts\python.exe
```

In [2]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import json
import math
import tempfile
import subprocess
import warnings
from pathlib import Path
```

```
import joblib
import librosa
import numpy as np
import pandas as pd
import tensorflow as tf

from scipy.stats import skew, kurtosis

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

```
Seed set to: 42
TensorFlow version: 2.20.0
```

In [3]:

```
# ============================================================
# 2. USER SETTINGS
# ============================================================
# MODE OPTIONS:
# - "external_file"      : use your own song/audio file
# - "existing_fma_track" : use a track_id from the FMA processed table

MODE = "external_file"

EXTERNAL_AUDIO_PATH = r"C:\Users\jdevo\Downloads\Queen - Bohemian Rhapsody
(Official Video Remastered).mp3"

TRACK_ID_TO_TEST = 568

# ============================================================
# SONG-LEVEL PREDICTION SETTINGS
# ============================================================

TOP_N_CANDIDATES = 15

# The genre with the highest average hybrid score becomes the main genre.
# Any other genre above this threshold is shown as a secondary/related prediction.
SECONDARY_GENRE_THRESHOLD = 0.42

# Used only for interpretation, not for forcing the final main prediction.
LOW_CONFIDENCE_THRESHOLD = 0.50

# ============================================================
# LOOPING AND WINDOW SETTINGS
# ============================================================
# If the audio is shorter than the required listening length, the notebook loops
it.
```

```
# If the audio is long enough, it uses windows spread across the song.

WINDOW_SECONDS = 15
MAX_WINDOWS = 4
MIN_WINDOW_VOTES = 2

LOOP_TARGET_SECONDS = WINDOW_SECONDS * MAX_WINDOWS
WINDOW_SELECTION_MODE = "evenly_spaced"   # "evenly_spaced" or "first_n"

# ============================================================
# STAGE 2 RARE-TAIL SAFETY SETTINGS
# ============================================================

REQUIRE_ANCHOR_PREDICTED = True
MIN_STAGE2_SCORE = 0.01
SUPPRESS_STAGE2_IF_LOW_CONFIDENCE = True

# ============================================================
# AUDIO SETTINGS
# ============================================================

SR = 22050
N_MELS = 64
N_FFT = 2048
HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((WINDOW_SECONDS * SR) / HOP_LENGTH)) + 1

print("MODE:", MODE)
print("EXTERNAL_AUDIO_PATH:", EXTERNAL_AUDIO_PATH)
print("TOP_N_CANDIDATES:", TOP_N_CANDIDATES)
print("SECONDARY_GENRE_THRESHOLD:", SECONDARY_GENRE_THRESHOLD)
print("LOW_CONFIDENCE_THRESHOLD:", LOW_CONFIDENCE_THRESHOLD)
print("WINDOW_SECONDS:", WINDOW_SECONDS)
print("MAX_WINDOWS:", MAX_WINDOWS)
print("MIN_WINDOW_VOTES:", MIN_WINDOW_VOTES)
print("LOOP_TARGET_SECONDS:", LOOP_TARGET_SECONDS)
print("WINDOW_SELECTION_MODE:", WINDOW_SELECTION_MODE)
print("MIN_STAGE2_SCORE:", MIN_STAGE2_SCORE)
print("SR:", SR)
print("N_MELS:", N_MELS)
print("MAX_FRAMES:", MAX_FRAMES)
```

```
MODE: external_file
EXTERNAL_AUDIO_PATH: C:\Users\jdevo\Downloads\Queen – Bohemian Rhapsody (Official
Video Remastered).mp3
TOP_N_CANDIDATES: 15
SECONDARY_GENRE_THRESHOLD: 0.42
LOW_CONFIDENCE_THRESHOLD: 0.5
WINDOW_SECONDS: 15
MAX_WINDOWS: 4
MIN_WINDOW_VOTES: 2
LOOP_TARGET_SECONDS: 60
WINDOW_SELECTION_MODE: evenly_spaced
MIN_STAGE2_SCORE: 0.01
SR: 22050
N_MELS: 64
MAX_FRAMES: 324
```

In [4]:

```
# ============================================================
# 3. LOAD FROZEN ARTIFACTS
# ============================================================

PROCESSED_DIR = "../data/processed"
MODELS_DIR = "../models"
RAW_METADATA_DIR = "../data/raw/metadata"

structured_model = joblib.load(
    f"{MODELS_DIR}/final_structured_multilabel_candidate150_best_model.joblib"
)

structured_scaler = joblib.load(
    f"{MODELS_DIR}/final_structured_multilabel_candidate150_scaler.joblib"
)

audio_model = tf.keras.models.load_model(
    f"{MODELS_DIR}/audio_multilabel_candidate150_expanded_final.keras"
)

candidate_label_cols = np.load(
    f"{PROCESSED_DIR}/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

with open(f"{PROCESSED_DIR}/final_project_frozen_config.json", "r") as f:
    final_project_config = json.load(f)

STAGE1_STRUCTURED_WEIGHT = float(final_project_config["stage1_structured_weight"])
STAGE1_AUDIO_WEIGHT = float(final_project_config["stage1_audio_weight"])
STAGE1_THRESHOLD = float(final_project_config["stage1_threshold"])

STAGE2_FINAL_STRATEGY = final_project_config["stage2_final_strategy"]
STAGE2_ANCHOR_TRIGGER_THRESHOLD =
float(final_project_config["stage2_anchor_trigger_threshold"])
STAGE2_TOP_K = int(final_project_config["stage2_top_k"])

rare_tail_router_df = pd.read_csv(
    f"{PROCESSED_DIR}/full161_rare_tail_routing_table.csv"
```

```
)

genre_inventory_df = pd.read_csv(
    f"{PROCESSED_DIR}/full_genre_inventory.csv"
)

full_master_df = pd.read_csv(
    f"{PROCESSED_DIR}/multilabel_full_master_table.csv"
)

features_reference = pd.read_csv(
    f"{RAW_METADATA_DIR}/features.csv",
    header=[0, 1, 2],
    index_col=0
)

print("Structured model loaded.")
print("Structured scaler loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Stage-1 model:", final_project_config["stage1_primary_system_name"])
print("Stage-1 weights:", STAGE1_STRUCTURED_WEIGHT, STAGE1_AUDIO_WEIGHT)
print("Stage-1 threshold:", STAGE1_THRESHOLD)
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Genre inventory shape:", genre_inventory_df.shape)
print("Full master shape:", full_master_df.shape)
print("Reference features shape:", features_reference.shape)
```

```
Structured model loaded.
Structured scaler loaded.
Audio model loaded.
Candidate labels: 150
Stage-1 model: Expanded Hybrid Global Threshold
Stage-1 weights: 0.1 0.9
Stage-1 threshold: 0.2
Rare-tail router shape: (13, 26)
Genre inventory shape: (163, 11)
Full master shape: (81574, 170)
Reference features shape: (106574, 518)
```

In [5]:

```
# ============================================================
# 4. PREPARE LOOKUPS AND HIERARCHY MAPS
# ============================================================

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)

genre_name_map = dict(
    zip(
        genre_inventory_df["genre_id"],
        genre_inventory_df["genre_name"]
    )
)

candidate_label_cols = [str(col) for col in candidate_label_cols]
```

```
candidate_label_ids = [
    int(col.replace("genre_", ""))
    for col in candidate_label_cols
]

candidate_id_to_index = {
    int(col.replace("genre_", "")): i
    for i, col in enumerate(candidate_label_cols)
}

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

inventory_only_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Inventory only"
].copy().reset_index(drop=True)

# Genre hierarchy map
genre_meta = {}

for _, row in genre_inventory_df.iterrows():
    gid = int(row["genre_id"])

    parent_id = None
    if not pd.isna(row["parent_id"]):
        parent_id = int(row["parent_id"])

    root_id = None
    if not pd.isna(row["root_genre_id"]):
        root_id = int(row["root_genre_id"])

    genre_meta[gid] = {
        "genre_name": row["genre_name"],
        "parent_id": parent_id,
        "parent_name": row["parent_name"] if not pd.isna(row["parent_name"]) else
None,
        "root_genre_id": root_id,
        "root_genre_name": row["root_genre_name"] if not
pd.isna(row["root_genre_name"]) else None
    }

# Flatten FMA feature columns
features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df.index = reference_feature_df.index.astype(int)
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
```

```
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)

reference_feature_means = reference_feature_df.mean(axis=0)
reference_feature_columns = list(reference_feature_df.columns)

full_master_indexed = full_master_df.set_index("track_id", drop=False)

fallback_rare_ids = fallback_router_df["rare_tail_genre_id"].astype(int).tolist()
fallback_rare_cols = [f"genre_{gid}" for gid in fallback_rare_ids]

print("Candidate genre labels:", len(candidate_label_ids))
print("Fallback rare-tail labels:", len(fallback_rare_ids))
print("Inventory-only labels:", inventory_only_df.shape[0])
print("Structured reference feature columns:", len(reference_feature_columns))
```

```
Candidate genre labels: 150
Fallback rare-tail labels: 10
Inventory-only labels: 3
Structured reference feature columns: 518
```

In [6]:

```
# ============================================================
# 5. RESOLVE INPUT AUDIO
# ============================================================

if MODE == "existing_fma_track":
    if TRACK_ID_TO_TEST not in full_master_indexed.index:
        raise ValueError(f"track_id {TRACK_ID_TO_TEST} not found in full master
table.")

    input_row = full_master_indexed.loc[TRACK_ID_TO_TEST]
    AUDIO_PATH = input_row["audio_path"]
    ACTIVE_TRACK_ID = int(input_row["track_id"])
    INPUT_SOURCE = "existing_fma_track"
    SONG_NAME = input_row["title"] if "title" in input_row.index else f"FMA Track
{ACTIVE_TRACK_ID}"

elif MODE == "external_file":
    AUDIO_PATH = EXTERNAL_AUDIO_PATH
    ACTIVE_TRACK_ID = None
    INPUT_SOURCE = "external_file"
    SONG_NAME = Path(EXTERNAL_AUDIO_PATH).stem

else:
    raise ValueError("MODE must be either 'external_file' or
'existing_fma_track'.")

print("Input source:", INPUT_SOURCE)
print("Song/audio name:", SONG_NAME)
print("Audio path:", AUDIO_PATH)
print("Track ID:", ACTIVE_TRACK_ID)
```

```
Input source: external_file
Song/audio name: Queen – Bohemian Rhapsody (Official Video Remastered)
Audio path: C:\Users\jdevo\Downloads\Queen – Bohemian Rhapsody (Official Video
Remastered).mp3
Track ID: None
```

In [7]:

```python
# ============================================================
# 6. HELPER FUNCTIONS
# ============================================================

def load_audio_robust(file_path, sr=SR):
    """
    Load audio robustly.

    Directly tries librosa first.
    If the file is an MP4/M4A-style container and fails, it tries ffmpeg.
    """
    try:
        y, sr_loaded = librosa.load(file_path, sr=sr, mono=True)

        if y is None or len(y) == 0:
            raise ValueError(f"Loaded audio is empty: {file_path}")

        return y, sr_loaded

    except Exception as first_error:
        ext = os.path.splitext(file_path)[1].lower()
        fallback_exts = {".mp4", ".m4a", ".aac", ".mov", ".3gp", ".webm"}

        if ext not in fallback_exts:
            raise RuntimeError(
                f"Could not load audio file: {file_path}\n"
                f"Original error: {first_error}"
            )

        try:
            with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp:
                tmp_wav = tmp.name

            cmd = [
                "ffmpeg",
                "-y",
                "-i",
                file_path,
                "-ac",
                "1",
                "-ar",
                str(sr),
                tmp_wav
            ]

            result = subprocess.run(
                cmd,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
```

```
                text=True
            )

            if result.returncode != 0:
                raise RuntimeError(result.stderr)

            y, sr_loaded = librosa.load(tmp_wav, sr=sr, mono=True)

            try:
                os.remove(tmp_wav)
            except Exception:
                pass

            if y is None or len(y) == 0:
                raise ValueError(f"Converted audio is empty: {file_path}")

            return y, sr_loaded

        except Exception as second_error:
            raise RuntimeError(
                f"Could not decode audio file: {file_path}\n\n"
                f"Direct librosa load error:\n{first_error}\n\n"
                f"FFmpeg fallback error:\n{second_error}\n\n"
                f"Recommended fix: convert the file to WAV first, then rerun the
pipeline."
            )


def loop_audio_if_needed(y, sr, target_seconds=LOOP_TARGET_SECONDS):
    """
    If audio is shorter than target_seconds, repeat/loop it until it reaches the
target length.
    If audio is long enough, return it unchanged.
    """
    target_len = int(target_seconds * sr)

    if len(y) >= target_len:
        return y, False

    repeats = math.ceil(target_len / len(y))
    y_looped = np.tile(y, repeats)[:target_len]

    return y_looped.astype(np.float32), True


def get_window_start_samples(
    y,
    sr=SR,
    window_seconds=WINDOW_SECONDS,
    max_windows=MAX_WINDOWS,
    mode=WINDOW_SELECTION_MODE
):
    """
    Select window starts across the audio.
    """
    total_len = len(y)
    window_len = int(window_seconds * sr)
```

```
    if total_len <= window_len or max_windows <= 1:
        return [0]

    max_start = total_len - window_len

    if mode == "first_n":
        starts = []
        current = 0

        while current <= max_start and len(starts) < max_windows:
            starts.append(int(current))
            current += window_len

        if len(starts) == 0:
            starts = [0]

        return starts

    if mode == "evenly_spaced":
        n_windows = min(max_windows, max(2, math.ceil(total_len / window_len)))
        starts = np.linspace(0, max_start, num=n_windows)
        starts = [int(x) for x in starts]
        starts = sorted(list(dict.fromkeys(starts)))[:max_windows]
        return starts

    raise ValueError("WINDOW_SELECTION_MODE must be 'evenly_spaced' or 'first_n'.")


def extract_window(y, start_sample, sr=SR, window_seconds=WINDOW_SECONDS):
    """
    Extract fixed length audio window.
    """
    window_len = int(window_seconds * sr)
    segment = y[start_sample:start_sample + window_len]

    if len(segment) < window_len:
        segment = np.pad(segment, (0, window_len - len(segment)), mode="constant")

    return segment.astype(np.float32)


def build_mel_input(
    y_segment,
    sr=SR,
    n_mels=N_MELS,
    n_fft=N_FFT,
    hop_length=HOP_LENGTH,
    max_frames=MAX_FRAMES
):
    """
    Convert audio window to Mel spectrogram input for CNN.
    """
    mel = librosa.feature.melspectrogram(
        y=y_segment,
        sr=sr,
        n_fft=n_fft,
```

```
        hop_length=hop_length,
        n_mels=n_mels
    )

    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

    if mel_db.shape[1] < max_frames:
        pad_width = max_frames - mel_db.shape[1]
        mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
    else:
        mel_db = mel_db[:, :max_frames]

    return mel_db.astype(np.float32)[None, :, :, None]


def safe_stat_vector(arr_2d, stat_name):
    """
    Compute statistics safely for structured branch.
    """
    arr_2d = np.asarray(arr_2d, dtype=np.float64)

    if arr_2d.ndim == 1:
        arr_2d = arr_2d.reshape(1, -1)

    if stat_name == "mean":
        out = np.mean(arr_2d, axis=1)
    elif stat_name == "std":
        out = np.std(arr_2d, axis=1)
    elif stat_name == "median":
        out = np.median(arr_2d, axis=1)
    elif stat_name == "min":
        out = np.min(arr_2d, axis=1)
    elif stat_name == "max":
        out = np.max(arr_2d, axis=1)
    elif stat_name == "skew":
        out = skew(arr_2d, axis=1, bias=False, nan_policy="omit")
    elif stat_name == "kurtosis":
        out = kurtosis(arr_2d, axis=1, bias=False, nan_policy="omit")
    else:
        raise ValueError(f"Unknown stat: {stat_name}")

    out = np.asarray(out, dtype=np.float64)
    out[~np.isfinite(out)] = 0.0

    return out.astype(np.float32)


def build_feature_matrices(y_segment, sr=SR):
    """
    Extract feature matrices used to approximate the FMA structured audio features.
    """
    y_segment = np.asarray(y_segment, dtype=np.float64)

    try:
        y_harmonic = librosa.effects.harmonic(y_segment)
    except Exception:
```

```
        y_harmonic = y_segment

    mats = {}
    mats["chroma_stft"] = librosa.feature.chroma_stft(y=y_segment, sr=sr)
    mats["chroma_cqt"] = librosa.feature.chroma_cqt(y=y_segment, sr=sr)
    mats["chroma_cens"] = librosa.feature.chroma_cens(y=y_segment, sr=sr)
    mats["tonnetz"] = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    mats["mfcc"] = librosa.feature.mfcc(y=y_segment, sr=sr, n_mfcc=20)
    mats["rms"] = librosa.feature.rms(y=y_segment)
    mats["spectral_centroid"] = librosa.feature.spectral_centroid(y=y_segment,
sr=sr)
    mats["spectral_bandwidth"] = librosa.feature.spectral_bandwidth(y=y_segment,
sr=sr)
    mats["spectral_contrast"] = librosa.feature.spectral_contrast(y=y_segment,
sr=sr)
    mats["spectral_rolloff"] = librosa.feature.spectral_rolloff(y=y_segment, sr=sr)
    mats["zcr"] = librosa.feature.zero_crossing_rate(y_segment)

    return mats


def build_structured_feature_vector(y_segment, reference_columns, reference_means,
sr=SR):
    """
    Build one structured feature vector from one audio window.
    """
    feature_mats = build_feature_matrices(y_segment, sr=sr)
    row_dict = {}

    for col in reference_columns:
        parts = col.split("_")

        try:
            component_idx = int(parts[-1]) - 1
            stat_name = parts[-2]
            feature_name = "_".join(parts[:-2])
        except Exception:
            row_dict[col] = np.nan
            continue

        if feature_name in feature_mats:
            mat = feature_mats[feature_name]
            stat_vec = safe_stat_vector(mat, stat_name)

            if 0 <= component_idx < len(stat_vec):
                row_dict[col] = float(stat_vec[component_idx])
            else:
                row_dict[col] = np.nan
        else:
            row_dict[col] = np.nan

    X_one = pd.DataFrame([row_dict], columns=reference_columns)
    X_one = X_one.replace([np.inf, -np.inf], np.nan)

    for col in reference_columns:
        if pd.isna(X_one.loc[0, col]):
            X_one.loc[0, col] = float(reference_means[col])
```

```
    return X_one.astype(np.float32)


def get_structured_scores(model, X_scaled):
    """
    Get structured model scores.
    """
    if hasattr(model, "decision_function"):
        scores = model.decision_function(X_scaled)
    elif hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_scaled)
    else:
        raise ValueError("Structured model supports neither decision_function nor
predict_proba.")

    return np.asarray(scores)


def scores_to_pseudoprobs(score_matrix):
    """
    Convert structured model raw scores to pseudo-probabilities.
    """
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))


def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    """
    Weighted average hybrid fusion.
    """
    return (w_structured * structured_probs) + (w_audio * audio_probs)


def confidence_level(score):
    """
    Convert score into a readable confidence category.
    """
    if score >= 0.70:
        return "High"
    elif score >= 0.50:
        return "Moderate"
    elif score >= 0.30:
        return "Low-to-moderate"
    else:
        return "Low"


def classify_relationship_to_main(genre_id, main_genre_id):
    """
    Explain whether a secondary genre is actually a subgenre/related genre
    based on the FMA hierarchy.
    """
    if genre_id == main_genre_id:
        return "Primary genre"

    g = genre_meta.get(genre_id, {})
```

```
    main = genre_meta.get(main_genre_id, {})

    g_parent = g.get("parent_id")
    g_root = g.get("root_genre_id")

    main_parent = main.get("parent_id")
    main_root = main.get("root_genre_id")

    if g_parent == main_genre_id or g_root == main_genre_id:
        return "Subgenre/child under primary genre"

    if main_parent == genre_id or main_root == genre_id:
        return "Broader parent of primary genre"

    if g_root is not None and main_root is not None and g_root == main_root:
        return "Related genre in same top-level family"

    return "Secondary associated genre"


def build_stage2_scores_baseline_prior(
    stage1_probs,
    stage1_pred,
    router_df,
    candidate_index_map,
    trigger_threshold,
    min_stage2_score,
    require_anchor_predicted=True
):
    """
    Safer Stage-2 rare-tail fallback scoring.
    """
    rows = []

    for _, row in router_df.iterrows():
        anchor_id = int(row["anchor_candidate_id"])
        anchor_idx = candidate_index_map[anchor_id]

        anchor_prob = float(stage1_probs[0, anchor_idx])
        anchor_predicted = int(stage1_pred[0, anchor_idx])

        p_anchor = 0.0 if pd.isna(row["p_rare_given_anchor"]) else
float(row["p_rare_given_anchor"])
        p_root = 0.0 if pd.isna(row["p_rare_given_root"]) else
float(row["p_rare_given_root"])

        prior_strength = max(p_anchor, p_root)
        stage2_score = anchor_prob * prior_strength

        passes_anchor_threshold = anchor_prob >= trigger_threshold
        passes_min_score = stage2_score >= min_stage2_score
        passes_anchor_predicted = (anchor_predicted == 1) if
require_anchor_predicted else True

        eligible = (
            passes_anchor_threshold and
            passes_min_score and
```

```
            passes_anchor_predicted
        )

        rows.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
            "anchor_candidate_name": row["anchor_candidate_name"],
            "anchor_prob": anchor_prob,
            "anchor_predicted": anchor_predicted,
            "prior_strength": prior_strength,
            "stage2_score": stage2_score,
            "eligible_stage2": eligible
        })

    all_scores_df = pd.DataFrame(rows).sort_values(
        ["eligible_stage2", "stage2_score", "anchor_prob"],
        ascending=[False, False, False]
    ).reset_index(drop=True)

    suggestions_df = all_scores_df[
        all_scores_df["eligible_stage2"] == True
    ].copy().reset_index(drop=True)

    return all_scores_df, suggestions_df
```

In [8]:

```
# ============================================================
# 7. LOAD AUDIO, LOOP IF NEEDED, AND BUILD WINDOWS
# ============================================================

y_original, sr_loaded = load_audio_robust(AUDIO_PATH, sr=SR)

original_duration = len(y_original) / sr_loaded

y_for_inference, audio_was_looped = loop_audio_if_needed(
    y_original,
    sr=sr_loaded,
    target_seconds=LOOP_TARGET_SECONDS
)

inference_duration = len(y_for_inference) / sr_loaded

window_starts = get_window_start_samples(
    y_for_inference,
    sr=sr_loaded,
    window_seconds=WINDOW_SECONDS,
    max_windows=MAX_WINDOWS,
    mode=WINDOW_SELECTION_MODE
)

window_info_rows = []
window_segments = []
```

```
for i, start_sample in enumerate(window_starts):
    segment = extract_window(
        y_for_inference,
        start_sample,
        sr=sr_loaded,
        window_seconds=WINDOW_SECONDS
    )

    start_sec = start_sample / sr_loaded
    end_sec = start_sec + WINDOW_SECONDS

    window_segments.append(segment)

    window_info_rows.append({
        "window_index": i,
        "start_sample": start_sample,
        "start_second": round(start_sec, 2),
        "end_second": round(end_sec, 2),
        "window_length_samples": len(segment)
    })

window_info_df = pd.DataFrame(window_info_rows)

print("Original audio duration seconds:", round(original_duration, 2))
print("Inference audio duration seconds:", round(inference_duration, 2))
print("Audio was looped:", audio_was_looped)
print("Number of windows:", len(window_segments))

display(window_info_df)
```

```
Original audio duration seconds: 359.45
Inference audio duration seconds: 359.45
Audio was looped: False
Number of windows: 4
```

	window_index	start_sample	start_second	end_second	window_length_samples
0	0	0	0.00	15.00	330750
1	1	2531678	114.82	129.82	330750
2	2	5063356	229.63	244.63	330750
3	3	7595034	344.45	359.45	330750

In [9]:

```
# ============================================================
# 8. RUN MODEL ON EACH WINDOW
# ============================================================

window_result_rows = []

structured_prob_list = []
audio_prob_list = []
hybrid_prob_list = []
```

```
for i, segment in enumerate(window_segments):
    print(f"Processing window {i + 1} of {len(window_segments)}...")

    X_audio_input = build_mel_input(
        segment,
        sr=sr_loaded,
        n_mels=N_MELS,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        max_frames=MAX_FRAMES
    )

    X_structured_one = build_structured_feature_vector(
        segment,
        reference_feature_columns,
        reference_feature_means,
        sr=sr_loaded
    )

    X_structured_scaled =
structured_scaler.transform(X_structured_one).astype(np.float32)

    structured_scores = get_structured_scores(
        structured_model,
        X_structured_scaled
    )

    structured_probs = scores_to_pseudoprobs(structured_scores)

    audio_probs = audio_model.predict(
        X_audio_input,
        verbose=0
    )

    hybrid_probs = fuse_probabilities(
        structured_probs,
        audio_probs,
        STAGE1_STRUCTURED_WEIGHT,
        STAGE1_AUDIO_WEIGHT
    )

    structured_prob_list.append(structured_probs[0])
    audio_prob_list.append(audio_probs[0])
    hybrid_prob_list.append(hybrid_probs[0])

    top_idx = int(np.argmax(hybrid_probs[0]))
    top_gid = candidate_label_ids[top_idx]
    top_name = genre_name_map.get(top_gid, str(top_gid))

    window_result_rows.append({
        "window_index": i,
        "top_genre_id": top_gid,
        "top_genre_name": top_name,
        "top_hybrid_confidence": float(hybrid_probs[0, top_idx]),
        "top_hybrid_confidence_percent": round(float(hybrid_probs[0, top_idx]) *
100, 2)
```

```
    })

window_results_df = pd.DataFrame(window_result_rows)

window_structured_probs = np.vstack(structured_prob_list)
window_audio_probs = np.vstack(audio_prob_list)
window_hybrid_probs = np.vstack(hybrid_prob_list)

stage1_probs = np.mean(
    window_hybrid_probs,
    axis=0,
    keepdims=True
)

print("Window structured probs shape:", window_structured_probs.shape)
print("Window audio probs shape:", window_audio_probs.shape)
print("Window hybrid probs shape:", window_hybrid_probs.shape)
print("Song-level averaged hybrid probs shape:", stage1_probs.shape)

print("Per-window top prediction summary:")
display(window_results_df)
```

```
Processing window 1 of 4...
Processing window 2 of 4...
Processing window 3 of 4...
Processing window 4 of 4...
Window structured probs shape: (4, 150)
Window audio probs shape: (4, 150)
Window hybrid probs shape: (4, 150)
Song-level averaged hybrid probs shape: (1, 150)
Per-window top prediction summary:
```

	window_index	top_genre_id	top_genre_name	top_hybrid_confidence	top_hybrid_confidence
0	0	38	Experimental	0.547312	
1	1	12	Rock	0.618163	
2	2	38	Experimental	0.521090	
3	3	38	Experimental	0.695340	

In [10]:

```
# ============================================================
# 9. AGGREGATE WINDOW RESULTS AND DETERMINE MAIN GENRE
# ============================================================

window_pred_matrix = (window_hybrid_probs >= STAGE1_THRESHOLD).astype(int)
required_window_votes = min(MIN_WINDOW_VOTES, len(window_segments))

aggregation_rows = []

for j, col in enumerate(candidate_label_cols):
    genre_id = int(col.replace("genre_", ""))
```

```
    genre_name = genre_name_map.get(genre_id, str(genre_id))

    structured_scores_for_genre = window_structured_probs[:, j]
    audio_scores_for_genre = window_audio_probs[:, j]
    hybrid_scores_for_genre = window_hybrid_probs[:, j]
    predictions_for_genre = window_pred_matrix[:, j]

    window_vote_count = int(predictions_for_genre.sum())
    window_vote_rate = window_vote_count / len(window_segments)

    mean_structured_score = float(np.mean(structured_scores_for_genre))
    mean_audio_score = float(np.mean(audio_scores_for_genre))
    mean_hybrid_score = float(np.mean(hybrid_scores_for_genre))

    max_hybrid_score = float(np.max(hybrid_scores_for_genre))
    min_hybrid_score = float(np.min(hybrid_scores_for_genre))

    technical_prediction = int(
        (mean_hybrid_score >= STAGE1_THRESHOLD) and
        (window_vote_count >= required_window_votes)
    )

    aggregation_rows.append({
        "genre_id": genre_id,
        "genre_name": genre_name,
        "structured_confidence_avg": mean_structured_score,
        "audio_confidence_avg": mean_audio_score,
        "hybrid_confidence_score": mean_hybrid_score,
        "hybrid_confidence_percent": round(mean_hybrid_score * 100, 2),
        "max_hybrid_confidence": max_hybrid_score,
        "min_hybrid_confidence": min_hybrid_score,
        "window_vote_count": window_vote_count,
        "window_vote_rate": window_vote_rate,
        "window_vote_percent": round(window_vote_rate * 100, 2),
        "required_window_votes": required_window_votes,
        "technical_prediction": technical_prediction
    })

candidate_results_df = pd.DataFrame(aggregation_rows).sort_values(
    ["hybrid_confidence_score", "window_vote_count"],
    ascending=[False, False]
).reset_index(drop=True)

main_prediction_row = candidate_results_df.iloc[0].copy()

MAIN_GENRE_ID = int(main_prediction_row["genre_id"])
MAIN_GENRE_NAME = main_prediction_row["genre_name"]
MAIN_CONFIDENCE = float(main_prediction_row["hybrid_confidence_score"])
MAIN_CONFIDENCE_PERCENT = round(MAIN_CONFIDENCE * 100, 2)
MAIN_CONFIDENCE_LEVEL = confidence_level(MAIN_CONFIDENCE)

low_confidence_flag = MAIN_CONFIDENCE < LOW_CONFIDENCE_THRESHOLD

print("Main song-level prediction:")
print("Genre:", MAIN_GENRE_NAME)
print("Confidence score:", MAIN_CONFIDENCE)
print("Confidence percent:", MAIN_CONFIDENCE_PERCENT, "%")
```

```
print("Confidence level:", MAIN_CONFIDENCE_LEVEL)
print("Low confidence flag:", low_confidence_flag)

print(f"Top {TOP_N_CANDIDATES} candidate genre scores:")
display(candidate_results_df.head(TOP_N_CANDIDATES))
```

```
Main song-level prediction:
Genre: Experimental
Confidence score: 0.5009398149414838
Confidence percent: 50.09 %
Confidence level: Moderate
Low confidence flag: False
Top 15 candidate genre scores:
```

	genre_id	genre_name	structured_confidence_avg	audio_confidence_avg	hybrid_confidence
0	38	Experimental	5.461522e-01	0.495916	0.5
1	12	Rock	4.977171e-01	0.352138	0.3
2	17	Folk	9.999993e-01	0.211177	0.2
3	250	Improv	7.500000e-01	0.117682	0.1
4	1	Avant-Garde	2.500227e-01	0.144652	0.1
5	10	Pop	9.611339e-09	0.141990	0.1
6	2	International	2.500000e-01	0.102103	0.1
7	74	Free-Jazz	7.500000e-01	0.036018	0.1
8	25	Punk	2.061154e-09	0.118094	0.1
9	32	Noise	5.006425e-01	0.060005	0.1
10	4	Jazz	4.998194e-01	0.058496	0.1
11	456	Minimalism	9.596827e-01	0.004503	0.1
12	27	Lo-Fi	2.034996e-04	0.109499	0.0
13	3	Blues	7.500000e-01	0.025040	0.0
14	20	Spoken	1.831820e-01	0.087729	0.0

In [11]:

```
# ============================================================
# 10. IDENTIFY SECONDARY / RELATED GENRE PREDICTIONS
# ============================================================
# The main genre is the highest scoring genre.
# Other genres above SECONDARY_GENRE_THRESHOLD are shown as secondary/related.
#
# Important:
# Not every secondary label is technically a subgenre of the main label.
# The relationship column explains whether it is:
# - an actual child/subgenre
```

```
# - in the same family
# - or simply another associated genre.

secondary_predictions_df = candidate_results_df[
    (candidate_results_df["genre_id"] != MAIN_GENRE_ID) &
    (candidate_results_df["hybrid_confidence_score"] >= SECONDARY_GENRE_THRESHOLD)
].copy().reset_index(drop=True)

if len(secondary_predictions_df) > 0:
    secondary_predictions_df["relationship_to_main"] =
secondary_predictions_df["genre_id"].apply(
        lambda gid: classify_relationship_to_main(int(gid), MAIN_GENRE_ID)
    )

    secondary_predictions_df["confidence_level"] =
secondary_predictions_df["hybrid_confidence_score"].apply(
        confidence_level
    )

else:
    secondary_predictions_df["relationship_to_main"] = []
    secondary_predictions_df["confidence_level"] = []

print("Secondary / related genre predictions above threshold:")
print("Secondary threshold:", SECONDARY_GENRE_THRESHOLD)

display(secondary_predictions_df)
```

```
Secondary / related genre predictions above threshold:
Secondary threshold: 0.42
```

genre_id	genre_name	structured_confidence_avg	audio_confidence_avg	hybrid_confidence_s

In [12]:

```
# ============================================================
# 11. CREATE USER-FACING SONG PREDICTION OUTPUT
# ============================================================

main_output_row = {
    "song_name": SONG_NAME,
    "prediction_type": "Main Genre",
    "genre_id": MAIN_GENRE_ID,
    "genre_name": MAIN_GENRE_NAME,
    "confidence_score": MAIN_CONFIDENCE,
    "confidence_percent": MAIN_CONFIDENCE_PERCENT,
    "confidence_level": MAIN_CONFIDENCE_LEVEL,
    "window_vote_count": int(main_prediction_row["window_vote_count"]),
    "window_vote_percent": float(main_prediction_row["window_vote_percent"]),
    "relationship_to_main": "Primary genre"
}

output_rows = [main_output_row]
```

```
for _, row in secondary_predictions_df.iterrows():
    output_rows.append({
        "song_name": SONG_NAME,
        "prediction_type": "Secondary / Related Genre",
        "genre_id": int(row["genre_id"]),
        "genre_name": row["genre_name"],
        "confidence_score": float(row["hybrid_confidence_score"]),
        "confidence_percent": float(row["hybrid_confidence_percent"]),
        "confidence_level": row["confidence_level"],
        "window_vote_count": int(row["window_vote_count"]),
        "window_vote_percent": float(row["window_vote_percent"]),
        "relationship_to_main": row["relationship_to_main"]
    })

song_prediction_output_df = pd.DataFrame(output_rows)

print("Final user-facing song prediction output:")
display(song_prediction_output_df)

print("\nReadable summary:")
print(f"The audio was predicted mainly as: {MAIN_GENRE_NAME}
({MAIN_CONFIDENCE_PERCENT}%).")

if low_confidence_flag:
    print("Note: This is flagged as low confidence because the top score is below
the low-confidence threshold.")

if len(secondary_predictions_df) > 0:
    print("Additional strong related predictions were also found:")
    for _, row in secondary_predictions_df.iterrows():
        print(
            f"- {row['genre_name']} "
            f"({row['hybrid_confidence_percent']}%) "
            f"[{row['relationship_to_main']}]"
        )
else:
    print("No secondary genres passed the secondary confidence threshold.")
```

```
Final user-facing song prediction output:
```

	song_name	prediction_type	genre_id	genre_name	confidence_score	confidence_percent
0	Queen – Bohemian Rhapsody (Official Video Rema...	Main Genre	38	Experimental	0.50094	50.09

```
Readable summary:
The audio was predicted mainly as: Experimental (50.09%).
No secondary genres passed the secondary confidence threshold.
```

In [13]:

```
# ============================================================
# 12. BUILD TECHNICAL STAGE-1 PREDICTION MATRIX
# ============================================================
# This is used for Stage 2 rare-tail fallback.
# It uses the technical prediction rule:
# score >= Stage-1 threshold AND enough window votes.

technical_predicted_candidate_df = candidate_results_df[
    candidate_results_df["technical_prediction"] == 1
].copy().reset_index(drop=True)

stage1_pred_final = np.zeros(
    (1, len(candidate_label_cols)),
    dtype=np.uint8
)

for _, row in technical_predicted_candidate_df.iterrows():
    genre_id = int(row["genre_id"])
    genre_index = candidate_id_to_index[genre_id]
    stage1_pred_final[0, genre_index] = 1

print("Technical Stage-1 predicted labels for rare-tail routing:")
display(technical_predicted_candidate_df)
```

```
Technical Stage-1 predicted labels for rare-tail routing:
```

	genre_id	genre_name	structured_confidence_avg	audio_confidence_avg	hybrid_confidence_
0	38	Experimental	0.546152	0.495916	0.5(
1	12	Rock	0.497717	0.352138	0.3(
2	17	Folk	0.999999	0.211177	0.2(

In [14]:

```
# ============================================================
# 13. SAFER STAGE-2 RARE-TAIL FALLBACK
# ============================================================

stage2_scores_df, stage2_suggestions_df = build_stage2_scores_baseline_prior(
    stage1_probs=stage1_probs,
    stage1_pred=stage1_pred_final,
    router_df=fallback_router_df,
    candidate_index_map=candidate_id_to_index,
    trigger_threshold=STAGE2_ANCHOR_TRIGGER_THRESHOLD,
    min_stage2_score=MIN_STAGE2_SCORE,
    require_anchor_predicted=REQUIRE_ANCHOR_PREDICTED
)

if SUPPRESS_STAGE2_IF_LOW_CONFIDENCE and low_confidence_flag:
    stage2_suggestions_df = stage2_suggestions_df.iloc[0:0].copy()

stage2_suggestions_df =
stage2_suggestions_df.head(STAGE2_TOP_K).copy().reset_index(drop=True)
```

```
print("All Stage-2 rare-tail scores:")
display(stage2_scores_df)

print("Final Stage-2 rare-tail suggestions:")
display(stage2_suggestions_df)
```

All Stage-2 rare-tail scores:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	ancho
0	1032	Turkish	102	Middle East	0.
1	189	Talk Radio	65	Radio	0.
2	176	Pacific	2	International	0.
3	374	Banter	20	Spoken	0.
4	465	Musical Theater	20	Spoken	0.
5	1060	Tango	46	Latin America	0.
6	173	N. Indian Traditional	86	Indian	0.
7	377	Deep Funk	19	Funk	0.
8	808	Salsa	46	Latin America	0.
9	493	Western Swing	651	Country & Western	0.

Final Stage-2 rare-tail suggestions:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	anchor

In [15]:

```
# ============================================================
# 14. INVENTORY-ONLY LABELS
# ============================================================

inventory_only_labels_df = inventory_only_df[
    [
        "rare_tail_genre_id",
        "rare_tail_genre_name",
        "anchor_candidate_name",
        "root_candidate_name",
        "fallback_mode"
    ]
].copy()

print("Inventory-only labels retained in taxonomy but not directly predicted:")
display(inventory_only_labels_df)
```

Inventory-only labels retained in taxonomy but not directly predicted:

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_name	root_candidate_name	fallb
0	174	South Indian Traditional	Indian	International	Inve
1	175	Bollywood	Indian	International	Inve
2	178	Be-Bop	Jazz	Jazz	Inve

In [16]:

```
# ============================================================
# 15. OPTIONAL FMA GROUND TRUTH CHECK
# ============================================================

ground_truth = None

if INPUT_SOURCE == "existing_fma_track" and ACTIVE_TRACK_ID is not None:
    row = full_master_indexed.loc[ACTIVE_TRACK_ID]

    true_candidate_ids = []

    for col in candidate_label_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    true_candidate_names = [
        genre_name_map.get(gid, str(gid))
        for gid in true_candidate_ids
    ]

    true_rare_ids = []

    for col in fallback_rare_cols:
        if col in row.index and int(row[col]) == 1:
            true_rare_ids.append(int(col.replace("genre_", "")))

    true_rare_names = [
        genre_name_map.get(gid, str(gid))
        for gid in true_rare_ids
    ]

    ground_truth = {
        "track_id": int(ACTIVE_TRACK_ID),
        "true_candidate_ids": true_candidate_ids,
        "true_candidate_names": true_candidate_names,
        "true_rare_tail_ids": true_rare_ids,
        "true_rare_tail_names": true_rare_names
    }

    print("Ground truth for selected FMA track:")
    print(json.dumps(ground_truth, indent=2))

else:
    print("No ground truth shown because this is an external file.")
```

No ground truth shown because this is an external file.

In [17]:

```
# ============================================================
# 16. FINAL SUMMARY JSON
# ============================================================

secondary_summary = []

for _, row in secondary_predictions_df.iterrows():
    secondary_summary.append({
        "genre_id": int(row["genre_id"]),
        "genre_name": row["genre_name"],
        "confidence_score": float(row["hybrid_confidence_score"]),
        "confidence_percent": float(row["hybrid_confidence_percent"]),
        "window_vote_count": int(row["window_vote_count"]),
        "relationship_to_main": row["relationship_to_main"]
    })

if len(stage2_suggestions_df) > 0:
    stage2_summary = []

    for _, row in stage2_suggestions_df.iterrows():
        stage2_summary.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_name": row["anchor_candidate_name"],
            "stage2_score": float(row["stage2_score"])
        })
else:
    stage2_summary = []

final_song_summary = {
    "song_name": SONG_NAME,
    "input_source": INPUT_SOURCE,
    "audio_path": AUDIO_PATH,
    "track_id": ACTIVE_TRACK_ID,

    "looping_and_windowing": {
        "original_duration_seconds": round(original_duration, 2),
        "inference_duration_seconds": round(inference_duration, 2),
        "audio_was_looped": bool(audio_was_looped),
        "window_seconds": WINDOW_SECONDS,
        "windows_used": len(window_segments),
        "min_window_votes": MIN_WINDOW_VOTES
    },

    "prediction_rule": {
        "main_genre_rule": "highest average hybrid confidence score",
        "secondary_genre_rule": f"other labels with confidence >=
{SECONDARY_GENRE_THRESHOLD}",
        "confidence_score_source": "averaged hybrid probability from structured and
audio branches",
        "low_confidence_threshold": LOW_CONFIDENCE_THRESHOLD
```

```
    },

    "main_prediction": {
        "genre_id": MAIN_GENRE_ID,
        "genre_name": MAIN_GENRE_NAME,
        "confidence_score": MAIN_CONFIDENCE,
        "confidence_percent": MAIN_CONFIDENCE_PERCENT,
        "confidence_level": MAIN_CONFIDENCE_LEVEL,
        "low_confidence_flag": bool(low_confidence_flag)
    },

    "secondary_predictions": secondary_summary,

    "stage2_rare_tail_suggestions": stage2_summary,

    "inventory_only_labels":
inventory_only_labels_df["rare_tail_genre_name"].tolist()
}

if ground_truth is not None:
    final_song_summary["ground_truth"] = ground_truth

print("Final song-level prediction summary:")
print(json.dumps(final_song_summary, indent=2))
```

```
Final song-level prediction summary:
{
  "song_name": "Queen \u2013 Bohemian Rhapsody (Official Video Remastered)",
  "input_source": "external_file",
  "audio_path": "C:\\Users\\jdevo\\Downloads\\Queen \u2013 Bohemian Rhapsody
(Official Video Remastered).mp3",
  "track_id": null,
  "looping_and_windowing": {
    "original_duration_seconds": 359.45,
    "inference_duration_seconds": 359.45,
    "audio_was_looped": false,
    "window_seconds": 15,
    "windows_used": 4,
    "min_window_votes": 2
  },
  "prediction_rule": {
    "main_genre_rule": "highest average hybrid confidence score",
    "secondary_genre_rule": "other labels with confidence >= 0.42",
    "confidence_score_source": "averaged hybrid probability from structured and audio
branches",
    "low_confidence_threshold": 0.5
  },
  "main_prediction": {
    "genre_id": 38,
    "genre_name": "Experimental",
    "confidence_score": 0.5009398149414838,
    "confidence_percent": 50.09,
    "confidence_level": "Moderate",
    "low_confidence_flag": false
  },
  "secondary_predictions": [],
  "stage2_rare_tail_suggestions": [],
  "inventory_only_labels": [
    "South Indian Traditional",
    "Bollywood",
    "Be-Bop"
  ]
}
```

In [18]:

```
# ============================================================
# 17. SAVE OUTPUTS
# ============================================================

os.makedirs(PROCESSED_DIR, exist_ok=True)

window_info_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_window_info.csv",
    index=False
)

window_results_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_window_top_predictions.csv",
    index=False
)
```

```
candidate_results_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_all_candidate_genre_scores.csv",
    index=False
)

song_prediction_output_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_final_song_prediction_output.csv",
    index=False
)

technical_predicted_candidate_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_technical_stage1_predictions.csv",
    index=False
)

secondary_predictions_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_secondary_predictions.csv",
    index=False
)

stage2_scores_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_stage2_all_scores.csv",
    index=False
)

stage2_suggestions_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_stage2_suggestions.csv",
    index=False
)

inventory_only_labels_df.to_csv(
    f"{PROCESSED_DIR}/notebook48_inventory_only_labels.csv",
    index=False
)

with open(f"{PROCESSED_DIR}/notebook48_final_song_prediction_summary.json", "w") as
f:
    json.dump(final_song_summary, f, indent=2)

print("Saved Notebook 48 song-level prediction outputs.")
```

```
Saved Notebook 48 song-level prediction outputs.
```

In [ ]:

```
# ============================================================
# 18. INTERPRETATION NOTES
# ============================================================

print("1. Notebook 48 is a song-level prediction notebook.")
print("2. The audio is looped only if it is shorter than the required inference
length.")
print("3. The model listens to multiple windows and averages the hybrid scores.")
print("4. The highest average hybrid score is reported as the main predicted
genre.")
print("5. Other labels above the secondary confidence threshold are reported as
```

```
secondary/related genres.")
print("6. These scores are model confidence scores, not guaranteed real-world
accuracy.")
print("7. If the top score is below the low-confidence threshold, the output should
be treated as uncertain.")
print("8. Stage 2 rare-tail labels remain fallback suggestions, not main
predictions.")
```

```
1. Notebook 48 is a song-level prediction notebook.
2. The audio is looped only if it is shorter than the required inference length.
3. The model listens to multiple windows and averages the hybrid scores.
4. The highest average hybrid score is reported as the main predicted genre.
5. Other labels above the secondary confidence threshold are reported as
secondary/related genres.
6. These scores are model confidence scores, not guaranteed real-world accuracy.
7. If the top score is below the low-confidence threshold, the output should be
treated as uncertain.
8. Stage 2 rare-tail labels remain fallback suggestions, not main predictions.
```

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the failure.

Click here for more info.

View Jupyter log for further details.